Assignment - 8

* Problem Statement
  write a C program to evaluate postfix expression using stacks.

* objectives:
  (1) To study stack and its operations.
  (2) To study the importance of expression evaluation

* Test Conditions

Test case	Input	Expected output
1	23+	5
2	23*	6
3	23+4*	20
4	52-3*	6

* Pseudo Code

EVALUATE_POSTFIX (expression)

1. Initialize an empty stack S.
2. Scan P from left to right
3. For each Symbol X in P:
   IF X is an operand THEN

PUSH X into S
ELSE IF X is an operator THEN
b ← POP(S)
a ← POP(S)
result ← a x b
PUSH result into S
END IF
5. answer ← POP(S)
6. Display answer
7. STOP.

* Time complexity

operation	Time Complexity
PUSH	O(1)
POP	O(1)
Postfix evaluation	O(n)

* Conclusion

Thus, implemented steps operations to evaluate the value of an expression.

* FAQ's

1. Explain how postfix/ prefix expression is evaluated.

→

- Postfix : Scan from left to right. Push operands into the stack. When an operator is found, pop two operands, perform the operation, and push the result.

- Prefix : Scan from right to left. Push operands into the stack. When an operater is found, pop two operands, perform the operation, and push the result.

2. What is ISP and ICP?

→

ISP (In Stack Priority): Priority of an operater when it is already present inside stack.

ICP (Incoming Priority): Priority of an operater when it is being inserted into the stack

Mainly used during infix to postfix/ prefix conversion.

3. Give various applications of Stack.

⇒

1. Expression conversion & evaluation.
2. Function call & recursion.
3. Parentheses matching.
4. Undo / Redo operations.
5. Backtracking.
6. Browser history.
7. Depth First Search (DFS).



PRACTICAL 8

EXPRESSION EVALUATION USING STACK

CSF20030 – Data Structures Laboratory

Experiment 08: Evaluate a Postfix Expression

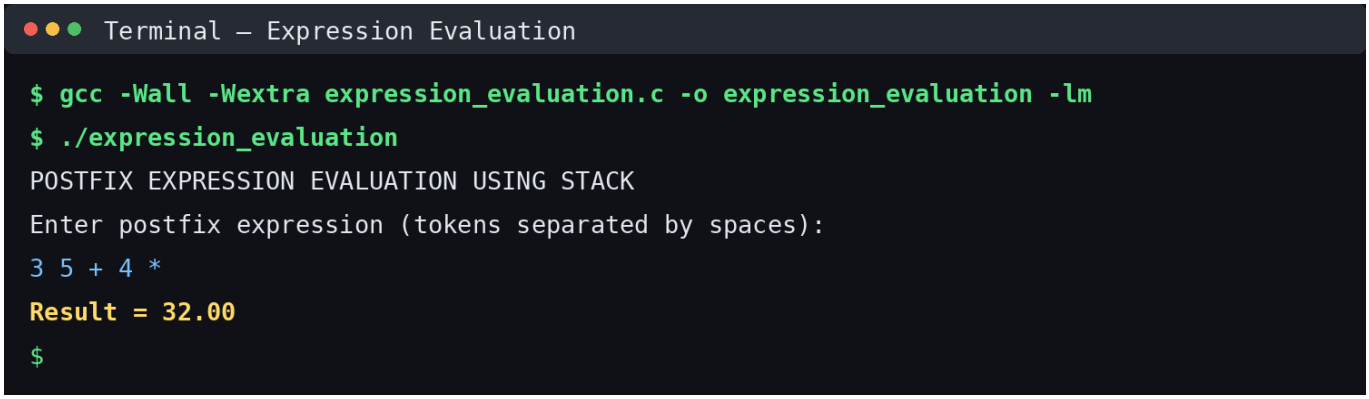
Aim: Write a C program to evaluate a postfix expression using a stack.

C Program

```
#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#define MAX 200
double stack[MAX];
int top = -1;
void push(double value) {
    if (top == MAX - 1) {
        printf("Error: Stack overflow.\n");
        exit(EXIT_FAILURE);
    }
    stack[++top] = value;
}
double pop(void) {
    if (top == -1) {
        printf("Error: Invalid postfix expression (stack underflow).\n");
        exit(EXIT_FAILURE);
    }
    return stack[top--];
}
/* Evaluate postfix expression with space-separated tokens.
   Supports multi-digit numbers and + - * / % ^ operators. */
double evaluatePostfix(char expression[]) {
    char *token = strtok(expression, " \t\n");
    while (token != NULL) {
        char *endptr;
        double value = strtod(token, &endptr);
        if (endptr != token && *endptr == '\0') {
            push(value);
        } else if (strlen(token) == 1 &&
                   strchr("+-*/%^", token[0]) != NULL) {
            double b = pop();
            double a = pop();
            switch (token[0]) {
                case '+': push(a + b); break;
                case '-': push(a - b); break;
                case '*': push(a * b); break;
                case '/':
                    if (b == 0) {
                        printf("Error: Division by zero.\n");
                        exit(EXIT_FAILURE);
                    }
                    push(a / b);
                    break;
                case '%':
                    if (b == 0) {
                        printf("Error: Modulo by zero.\n");
                        exit(EXIT_FAILURE);
                    }
                    push(fmod(a, b));
                    break;
                case '^': push(pow(a, b)); break;
            }
        } else {
            printf("Error: Invalid token '%s'.\n", token);
            exit(EXIT_FAILURE);
        }
        token = strtok(NULL, " \t\n");
    }
    if (top != 0) {
        printf("Error: Invalid postfix expression.\n");
        exit(EXIT_FAILURE);
    }
    return pop();
}
int main(void) {
    char expression[500];
    printf("POSTFIX EXPRESSION EVALUATION USING STACK\n");
    printf("Enter postfix expression (tokens separated by spaces):\n");
    if (fgets(expression, sizeof(expression), stdin) == NULL) {
        printf("Input error.\n");
        return 1;
    }
```

```
    top = -1;
    double result = evaluatePostfix(expression);
    printf("Result = %.2f\n", result);
    return 0;
}
```

Sample Input and Output Screenshot



Input: 3 5 + 4 *
Output: Result = 32.00
The expression is evaluated as (3 + 5) × 4 = 32.